Make a Static Webpage

CS 407 Intro To Linux

November 4, 2025

1 Introduction

In this lab you will write your first website. The website will be made with:

1. HTML - Language for the website content.
2. CSS - Cascading Style Sheets. Used for styling the website.
3. Javascript - Language used for adding some interactivity to the website.
4. Apache2 - The webserver software that will handle the details of HTTP/HTTPS for us.
5. Debian 13 - The operating system we'll run on
6. vim - The text editor you'll use to write the code.

In just a few minutes you'll have a real website on the internet! You'll be able to send a link to your website to friends and family to show off your work.

2 Setup

First you need to set up your machine. ssh into a Debian 13 droplet and run the following commands:

```
root@machine$ apt install vim apache2
```

When you install apache2, it will start running a default web page. Go to the ip address of your droplet in your browser on your phone and pc. You'll see the default page!

TODO add picture here

Now let's change the webpage really quick to make sure things are working. The webpage being shown to you, by default, is in **/var/www/html**. Let's change it and restart apache.

```
root@machine$ cd /var/www/html
root@machine$ ls
# you should see index.html
root@machine$ rm index.html
root@machine$ vim index.html
# create a new file. Make the file say "I am learning Linux"
root@machine$ cat index.html
```

```
I am learning linux.
root@machine$ service apache2 restart
```

Now go to your droplet ip address in your browser. You should now see ”I am learning Linux” and NOT the default page. Congratulations!

Note! Sometimes the browser will cache old versions of the webpage. If you are still seeing the default apache page, try the following:

1. Make absolutely sure that you followed the instructions above!! Are you sure you did everything?
2. You can force clear the cache (google for how, I think its something like CTRL + F5).
3. restart your browser
4. try a private browser window
5. try accessing the webpage on another device.

3 Your first HTML

HTML is *Hyper Text Markup Language*. Its the language you use for specifying the content like words, images, etc. that are shown on your website.

3.1 HTML Summary

HTML has a handful of useful tags for specifying the content of your page. For example you have:

1. h1 - header 1
2. h2 - header 2. A little smaller than header 1.
3. p - a paragraph
4. html - the tags that goes around the whole html
5. head - header infomation
6. body - the body of the webpage. The content the user will see.

You have to use

3.2 index.html

Let's replace index.html with some proper html instead of that text we put before. Open **/var/www/html/index.html** and put the following text:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Linux Class</title>
</head>
<body>
    <h1>About Me!</h1>
    <p>My name is Fulano and I'm learning Linux.</p>
    <h2>What I know</h2>
    <p>I know alot about Linux</p>
</body>
</html>
```

Then restart apache2 and reload your page in the browser. Do you see the new webpage?

```
root@machine$ service apache2 restart
```

TODO add picture here

4 A Website With 2 Pages

Modify **index.html** so that it has a link to another page. The other page is called **linuxdetails.html** See below for the file contents. Make sure you save both files in **/var/www/html**.

4.1 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Linux Class</title>
</head>
<body>
    <h1>About Me!</h1>
    <p>My name is Fulano and I'm learning Linux.</p>
    <h2>I know linux</h2>
    <p><a href="linuxdetails.html">Click here for more information</a></p>
</body>
</html>
```

4.2 linuxdetails.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Linux Details</title>
</head>
<body>
    <h1>What I know</h1>
    <p>Here are some things I know about Linux.</p>
    <h2>mkdir</h2>
    <p>Create a directory</p>
    <h2>cp</h2>
    <p>Copy a file</p>
    <h2>mv</h2>
    <p>Rename a file</p>
    <h2>rm</h2>
    <p>Delete a file</p>
</body>
</html>
```

4.3 Make it Better!

At this point you know alot about Linux. Expand your **linuxdetails.html** page so it says 10 things you know. For inspiration, consider the following:

- cp
- mkdir
- mv
- rm
- cd
- grep
- pipes
- awk
- touch
- echo
- vim
- bash
- ssh
- sftp

4.4 Can you Add a Third Page?

Why not add another link to index.html like this:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Linux Class</title>
</head>
<body>
    <h1>About Me!</h1>
    <p>My name is Fulano and I'm learning Linux.</p>
    <h2>I know linux</h2>
    <p><a href="linuxdetails.html">Click here for more information</a></p>
    <h2>I also know how to cook</h2>
    <p><a href="cooking.html">Click here for my recipes</a></p>
</body>
</html>
```

and then add another file **cooking.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Recipes</title>
</head>
```

```html
<body>
    <h1>My recipe</h1>
    We will make grilled cheese.
    <h2>Grilled cheese</h2>
    <ol>
    <li>First take 2 slices of white bread and put mayonnaise on one side of each.</li>
    <li>Put 2 slices of your favorite cheese on the mayonnaise side of one bread.</li>
    <li>Place the other slice of bread mayo side down upon the cheese.</li>
        <li>Put more mayo on the outside of one piece of bread.</li>
        <li>Place the bread mayoside down onto a warm pan.</li>
        <li>All stoves are different. Experiment with the heat. You want the bread to start getting golden brown and the cheese to melt.</li>
        <li>When one side is toasty and golden brown, apply mayo to the other side and flip the sandwich.</li>
        <li>When this side is toasty and golden brown, remove from the pan. The cheese should be melted.</li>
    </ol>
    <h2>What can go wrong</h2>
    <p> If the fire is too hot, you can burn the bread before the cheese melts.</p>
    <p> Also, some types of cheese do not melt very well. American cheese, cheddar, provolone and swiss will melt well. Muenster cheese, on the other hand, might not melt.</p>
    <h2>Enjoy!</h2>
    <p> Enjoy your sandwich! If it burned, you can always toss it out and try again, its just bread and a couple of slices of cheese.</p>
</body>
</html>
```

5 Adding CSS

CSS is *Cascading Style Sheets* and is the language used for specifying the style of your webpage. You can do quite a bit with CSS! For now we'll just use it to change the page colors.

Add a file called **style.css** to **/var/www/html** and add a link to it in the headers of your html pages. Below you can find an example of how **index.html** has the stylesheet in the header.

5.1 style.css

```
body {
    background-color: #F5F5F5;
    color: #0A0A11;
}

h1 {
    color: #0108A3;
}

h2 {
    color: #0008EE;
}
```

5.2 index.html

Note how in the page head you now see a link to style.css

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="style.css">
    <title>Linux Class</title>
</head>
<body>
    <h1>About Me!</h1>
    <p>My name is Fulano and I'm learning Linux.</p>
    <h2>I know linux</h2>
    <p><a href="linuxdetails.html">Click here for more information</a></p>
    <h2>I also know how to cook</h2>
    <p><a href="cooking.html">Click here for my recipes</a></p>
</body>
</html>
```

6 Javascript

And now we will add Javascript. Javascript is a language that allows you to add some functionality to the webpage. Some people like to write websites that don't use javascript, but perform all functionality on the server aka the ”backend”. Other people like to write javascript-heavy websites that do most of their work in the ”frontend”. In this example you'll get a peak at what javascript looks like.

Add a file called **counter.js** to **/var/www/html** and then modify **index.html** so it incorporates the javascript.

6.1 counter.js

```
let clickCount = 0;

function countClicks() {
    clickCount++;
    document.getElementById('clickDisplay').innerText =
         `Button clicked ${clickCount} time${clickCount === 1 ? '' : 's'}`;
}
```

6.2 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="style.css">
    <title>Linux Class</title>
</head>
<body>
        <h1>About Me!</h1>
        <p>My name is Fulano and I'm learning Linux.</p>

        <h2>I know Linux</h2>
        <p><a href="linuxdetails.html">Click here for more information</a></p>

        <h2>I also know how to cook</h2>
        <p><a href="cooking.html">Click here for my recipes</a></p>

        <h2>Click Counter</h2>
        <button onclick="countClicks()">Click me!</button>
        <p id="clickDisplay">Button clicked 0 times</p>
        <script src="counter.js" defer></script>

</body>
</html>
```

7 Congratulations!

You reached the end! You were able to deploy a real static website on Debian Linux with the Apache2 webserver! Pat yourself on the back and feel proud! But know, this is just the tip of the iceberg. There's still a ton to learn.

While you wait for the next lesson, you should look into more about html. There are many more tags besides h1, h2, p, and a. And CSS can do many more things than you've seen.